인터프리터 스터디 2 - 파싱편

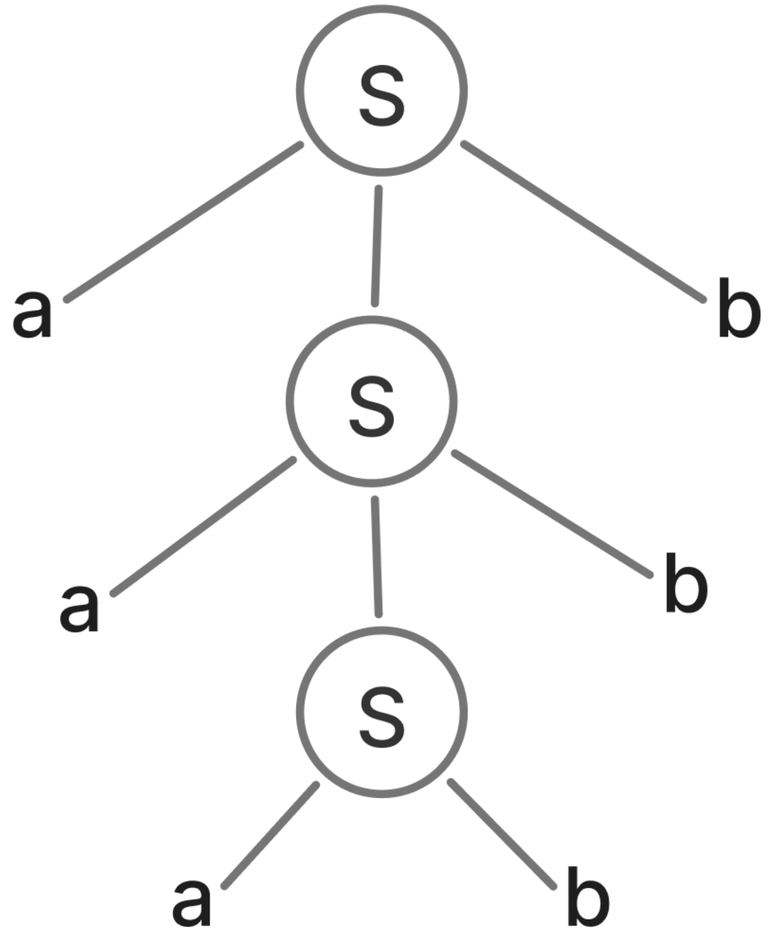
2026 김은빈

구문 분석(파싱)의 종류

생성 규칙

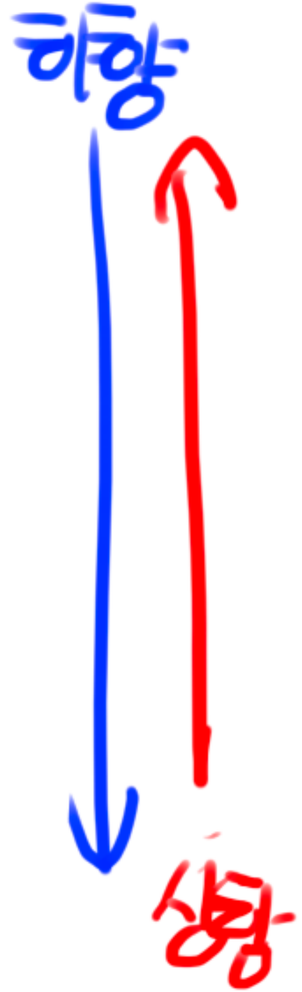
S → ab
S → aSb

S → aSb



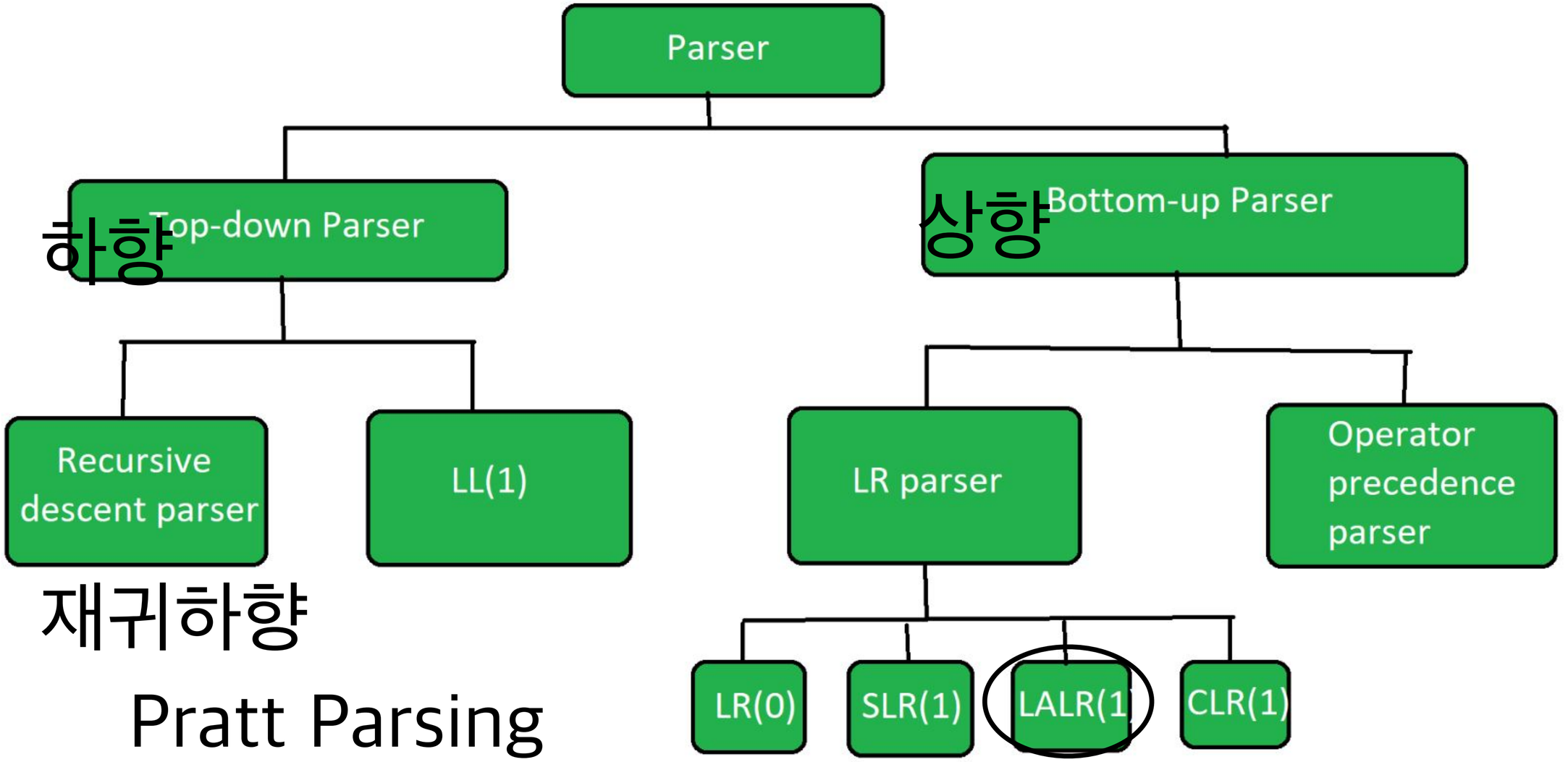
S → aSb

S → ab



결과: aaabbb

구문 분석(파싱)의 종류



Pratt Parsing

https://www.geeksforgeeks.org/compiler-design/types-of-parsers-in-compiler-design/

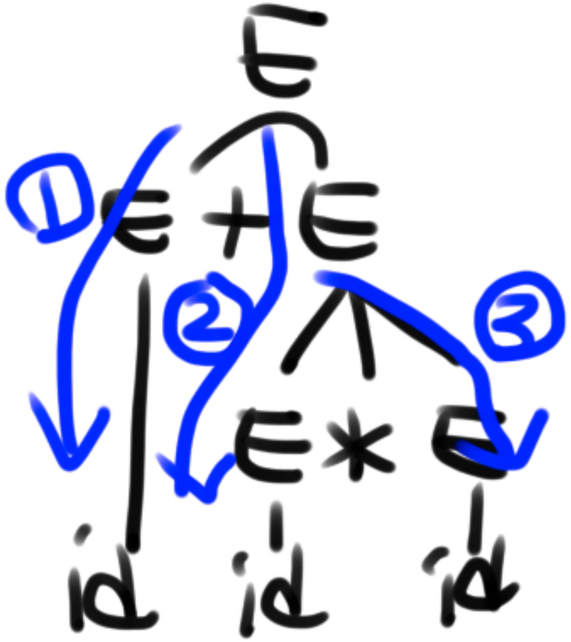
좌단(Leftmost) 우단(Rightmost)

E
⇒ **E** O E
⇒ (**E**) O E
⇒ (**E** O E) O E
⇒ (id **O** E) O E
⇒ (id + **E**) O E
⇒ (id + id) **O** E
⇒ (id + id) * **E**
⇒ (id + id) * id

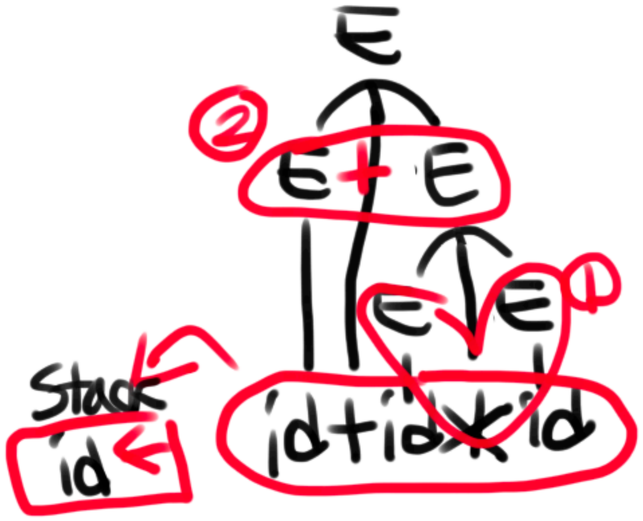
E
⇒ E O **E**
⇒ E **O** id
⇒ **E** * id
⇒ (**E**) * id
⇒ (E O **E**) * id
⇒ (E **O** id) * id
⇒ (**E** + id) * id
⇒ (id + id) * id

하향(LL), 상향(LR)인 이유

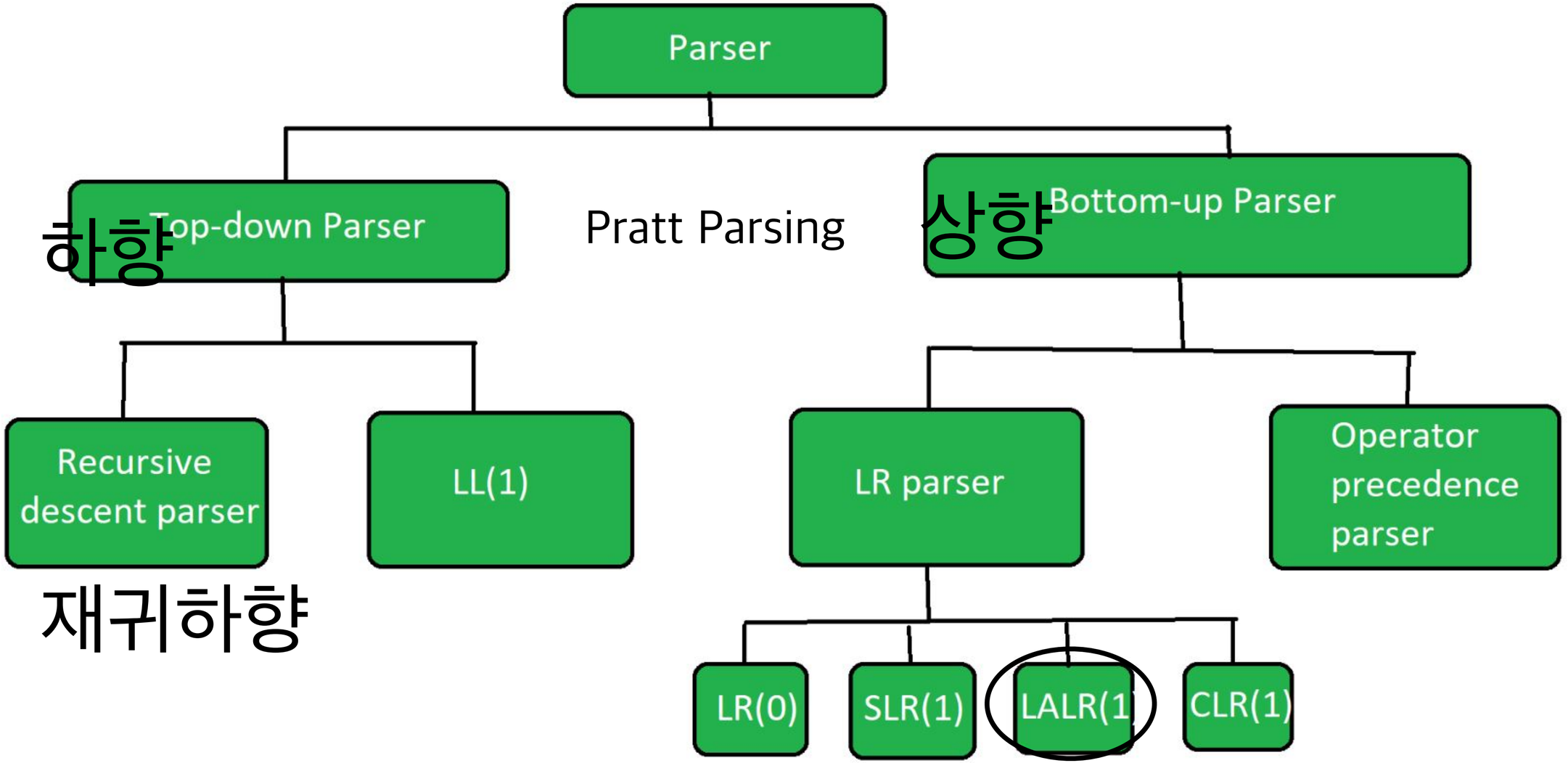
LL(k) **L**eft to right 스캔
Leftmost 유도



LR(k) **L**eft to right 스캔
Rightmost 유도 (역순으로)



구문 분석(파싱)의 종류



https://www.geeksforgeeks.org/compiler-design/types-of-parsers-in-compiler-design/

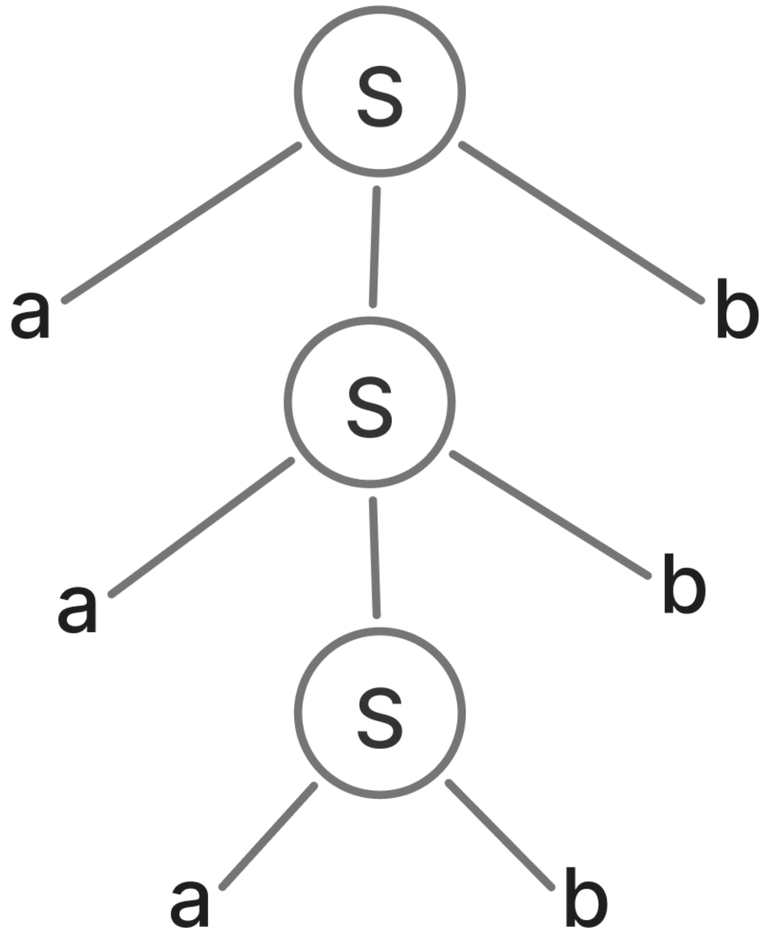
하향식 파싱
(Top-Down Parsing)

재귀 하강(Recursive descent)

S → aSb

S → aSb

S → ab



재귀 하강(Recursive descent)

Left Recursion

1 - 2 - 3

S: S '-' term | term

S에서 계속 재귀로 빠짐 …
Left Recursion 문제

재귀 하강(Recursive descent)

Left Recursion 제거

1 - 2 - 3

S: S '-' term | term

 S: term ('-' term)*

재귀 하강(Recursive descent)

Left Recursion 제거

1 - 2 - 3

S: S '-' term | term



S: term ('-' term)* e.g. 1-2-3

((1-2)-3) 같은 좌결합 트리를
따로 만들어줘야함

RustPython(ruff)도
일부 이 방식으로 문제 해결

CPython의 해결법: seed growing

엄밀하게는 Packrat parser + seed growing

Packrat parser = **P**arsing **E**xpression **G**rammer
(CFG가 아닌)

PEG는 그냥 1번 규칙 or 2번 규칙일 때
왼쪽에 있는거 먼저 따르는거임

https://en.wikipedia.org/wiki/Packrat_parser

Parsers Can Support Left Recursion: https://web.cs.ucla.edu/~todd/research/pepm08.pdf

CPython의 해결법: seed growing

```
S: S '-' term | term
term: NUMBER

# 입력
1 - 2 - 3
```

CPython의 해결법: seed growing

```
S: S '-' term | term
term: NUMBER
```

```
# 입력
1 …
```

```
S: S(이건 없으니까 선택하지 마셈) '-' term | term
```

CPython의 해결법: seed growing

```
S: S '-' term | term
term: NUMBER
```

```
# 입력
1 - 2 ...
```

```
S: S(1) '-' term | term
```

CPython의 해결법: seed growing

```
S: S '-' term | term
term: NUMBER
```

```
# 입력
1 - 2 - 3
```

```
S: S(1-2) '-' term | term
```

CPython의 해결법: seed growing

S: S '-' term | term
term: NUMBER

입력
1 - 2 - 3 ㅎㅎ

결과: ((1 - 2) - 3)



LGTM!

```python
def memoize_left_rec_wrapper(self: "Parser") -> T | None:
        mark = self._mark()
        key = mark, method_name, ()
        # Fast path: cache hit, and not verbose.
        if key in self._cache and not self._verbose:
            tree, endmark = self._cache[key]
            self._reset(endmark)
            return tree
        # Slow path: no cache hit, or verbose.
        verbose = self._verbose
        fill = "  " * self._level
        if key not in self._cache:
            if verbose:
                print(f"{fill}{method_name} ... (looking at {self.showpeek()})")
            self._level += 1

            # For left-recursive rules we manipulate the cache and
            # loop until the rule shows no progress, then pick the
            # previous result.  For an explanation why this works, see
            # https://github.com/PhilippeSigaud/Pegged/wiki/Left-Recursion
            # (But we use the memoization cache instead of a static
            # variable; perhaps this is similar to a paper by Warth et al.
            # (http://web.cs.ucla.edu/~todd/research/pub.php?id=pepm08).

            # Prime the cache with a failure.
            self._cache[key] = None, mark
            lastresult, lastmark = None, mark
            depth = 0
            if verbose:
                print(f"{fill}Recursive {method_name} at {mark} depth {depth}")
```

: seed growing

씨앗을 심는 코드를 살펴보자

좌재귀 부분에 대한 캐시를
None(실패)로 미리 지정



https://github.com/python/cpython/blob/main/Tools/peg_generator/pegen/parser.py

CPython의 해결법: seed growing

```
        while True:
            self._reset(mark)
            self.in_recursive_rule += 1
            try:
                result = method(self)
            finally:
                self.in_recursive_rule -= 1
            endmark = self._mark()
            depth += 1
            if not result:
                if verbose:
                    print(f"{fill}Fail with {lastresult!s:.200} to {lastmark}")
                break
            if endmark ≤ lastmark:
                if verbose:
                    print(f"{fill}Bailing with {lastresult!s:.200} to {lastmark}"
                break
            self._cache[key] = lastresult, lastmark = result, endmark
```

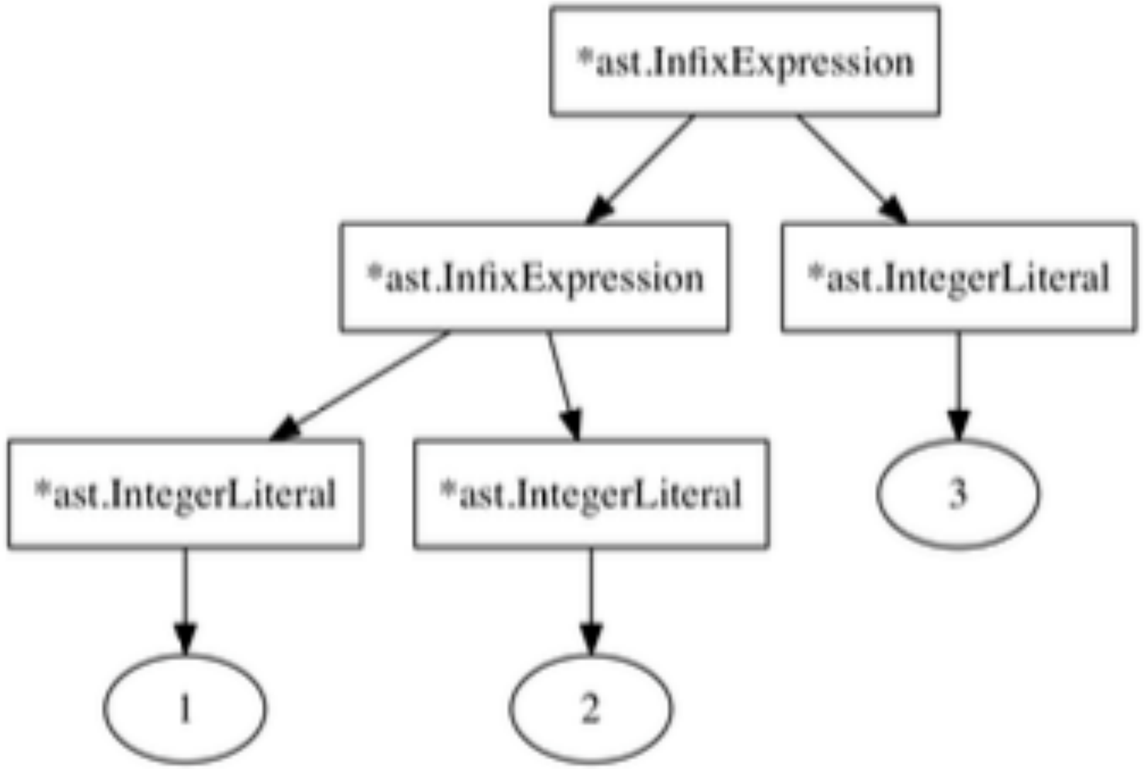
반복문이 끝나는 조건을
살펴보자
(더이상 길어지지 않을 때)



https://github.com/python/cpython/blob/main/Tools/peg_generator/pegen/parser.py

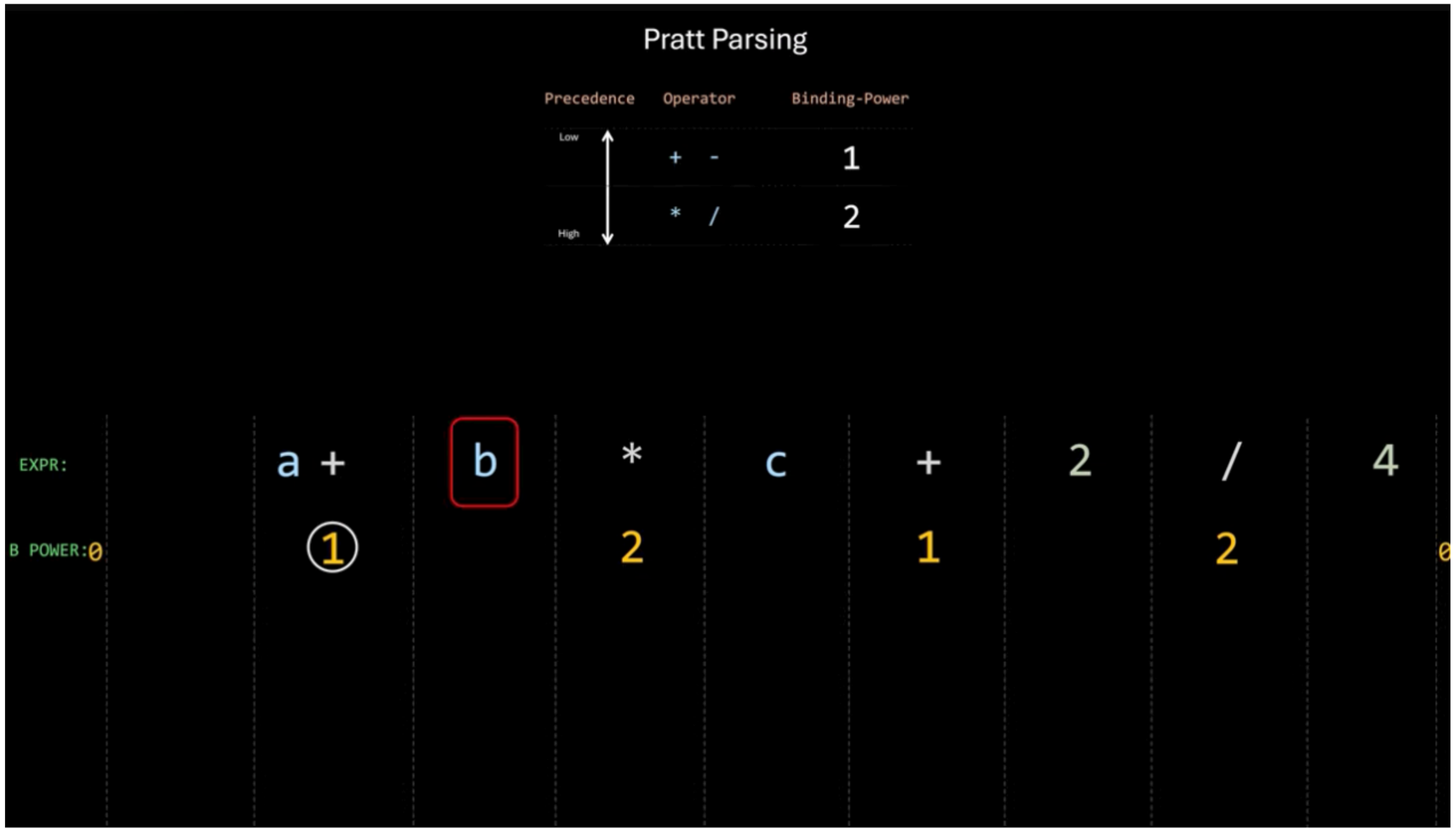
Pratt Parsing

하향식 연산자 우선순위 (Top-Down Operator Preceedence)




```
1    +    2    +    3    ;
     ↑    ↑
     |    p.peekToken
p.curToken
```

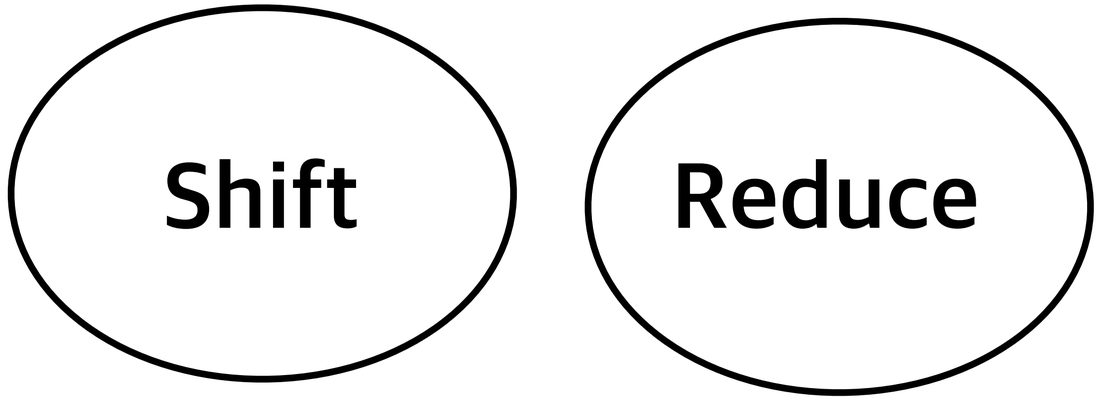
Pratt Parsing



https://youtu.be/0c8b7YfsBKs?t=326

상향식 파싱
(Bottom-Up Parsing)

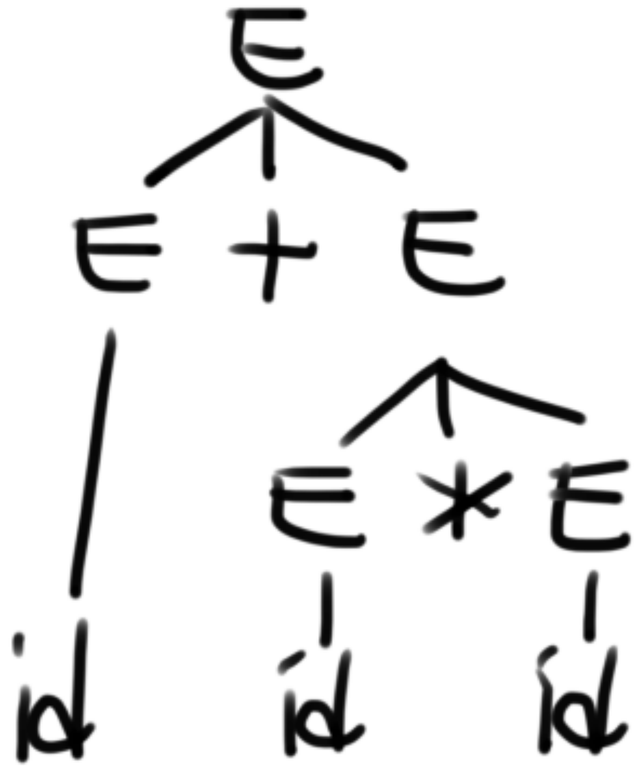
상향식 (shift - reduce, LR)



상향식은 이 두가지로 설명됨

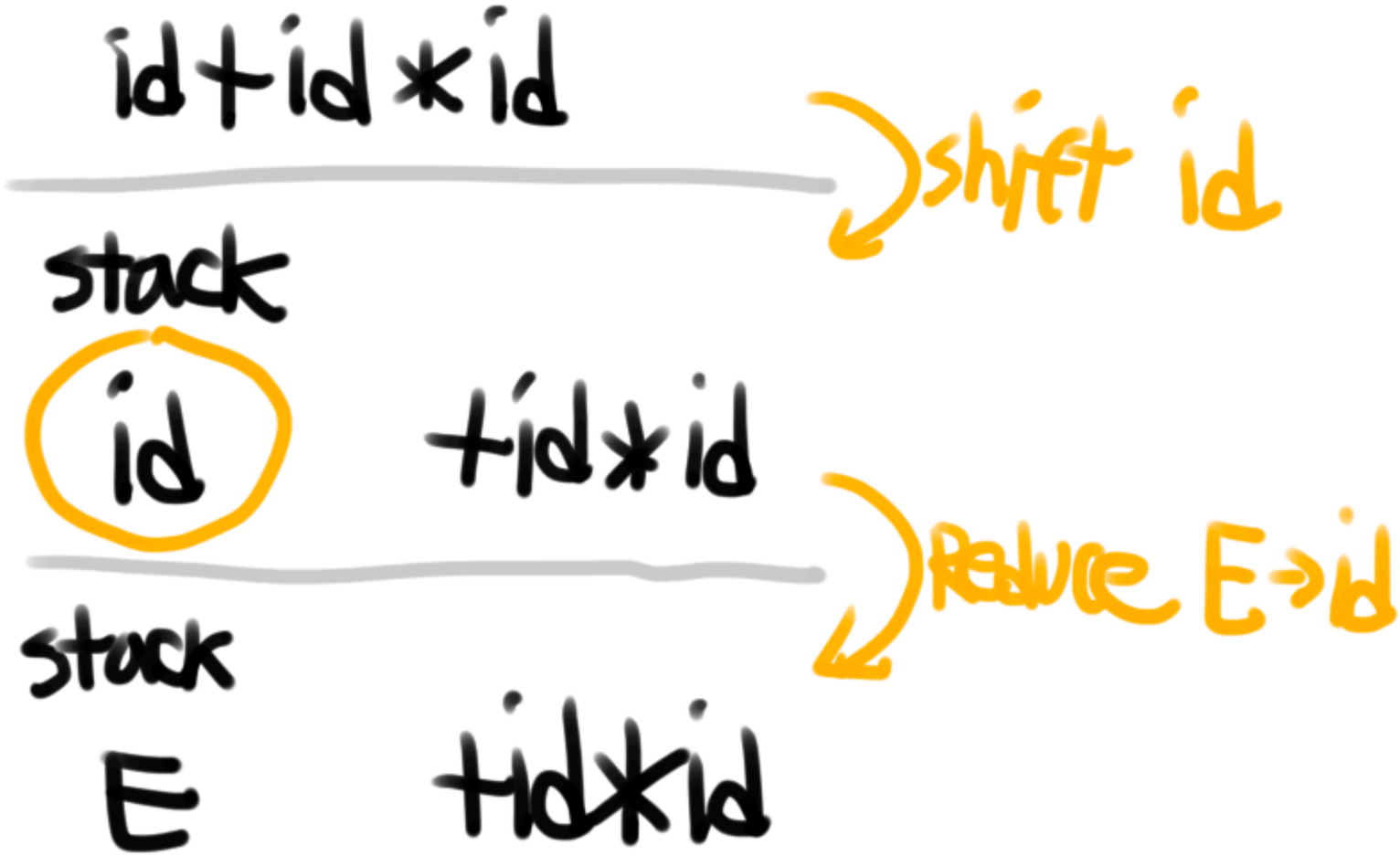
상향식 (shift - reduce, LR)

$$E \rightarrow E + E$$
$$E \rightarrow E * E$$
$$E \rightarrow id$$

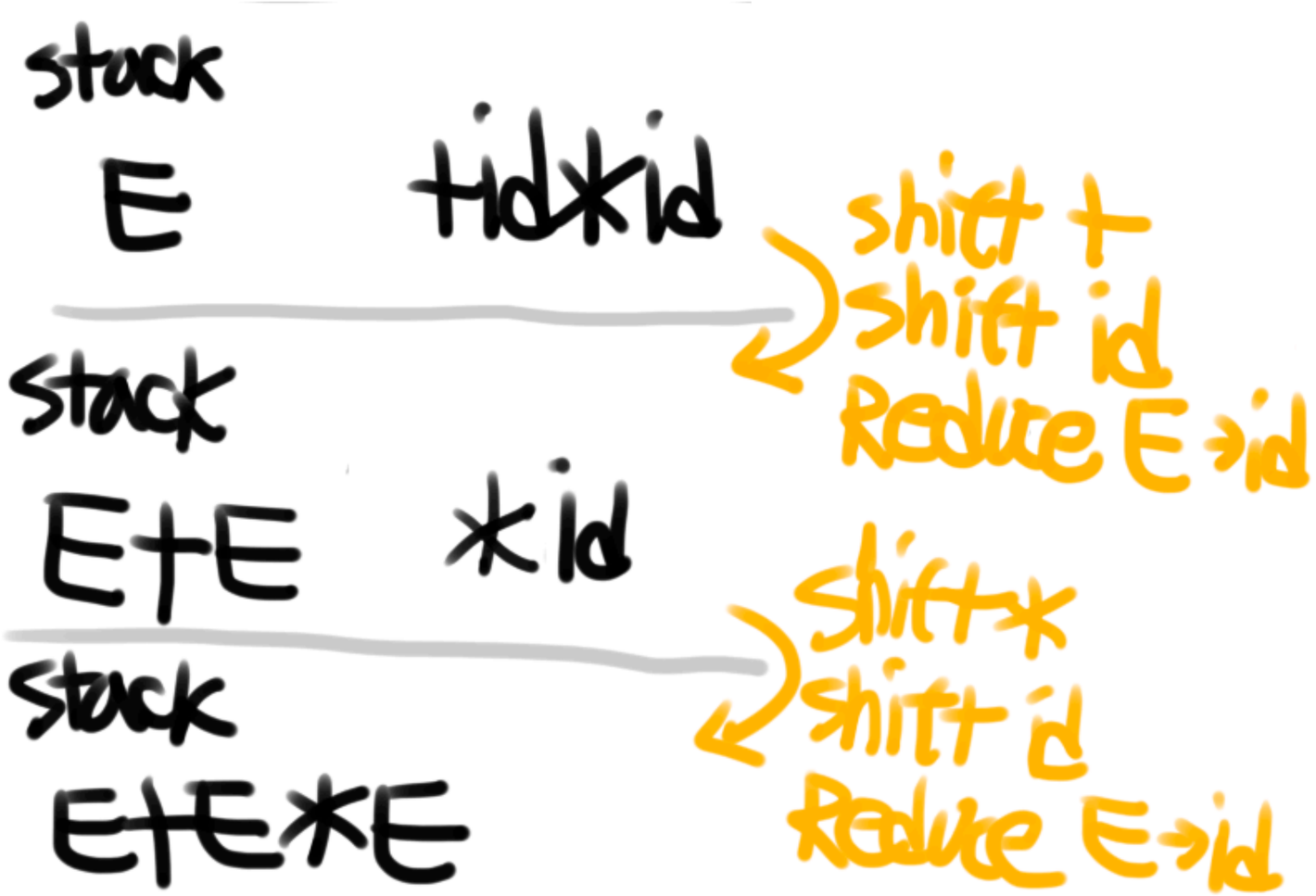


e.g id + id * id

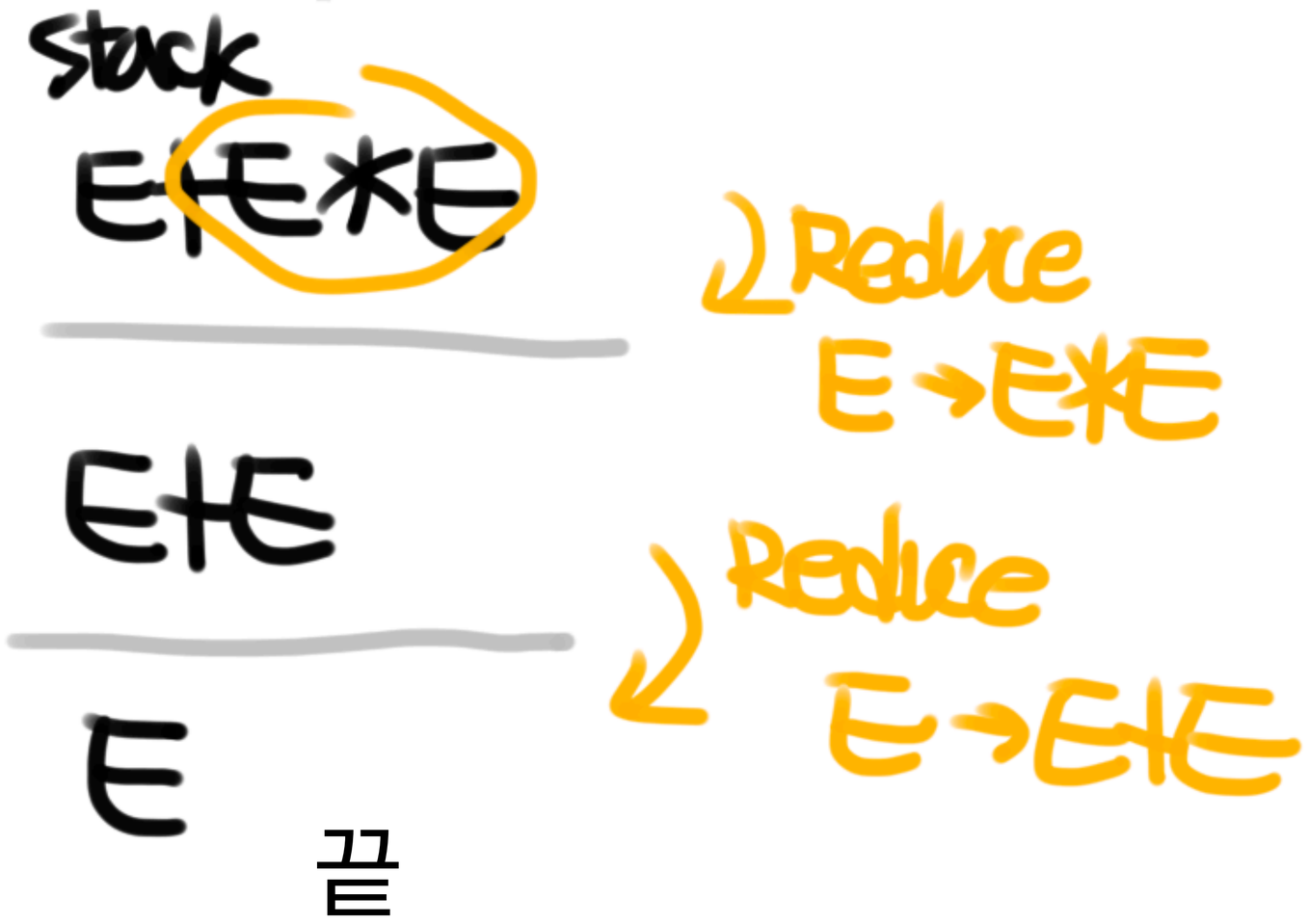
상향식 (shift - reduce, LR)



상향식 (shift - reduce, LR)

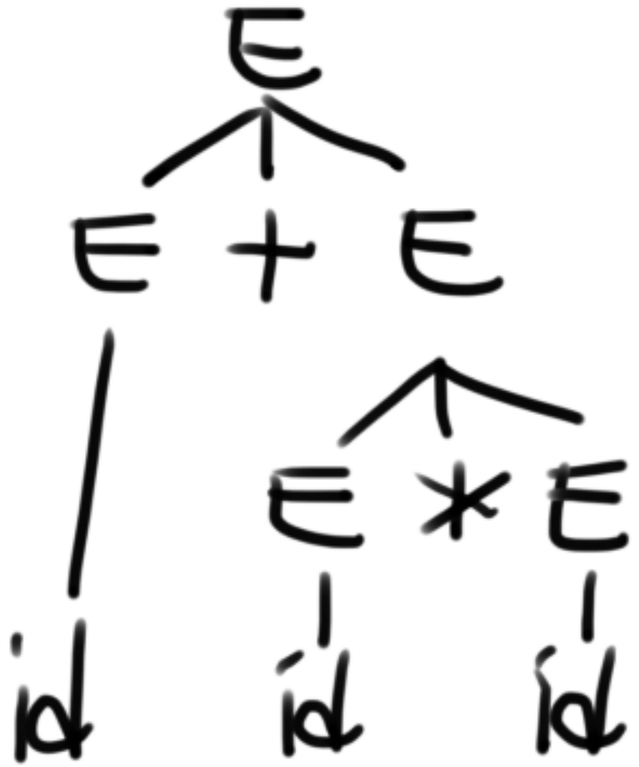


상향식 (shift - reduce, LR)



상향식 (shift - reduce, LR)

단계	스택	입력	구문분석행동
0	$	id+id*id$	shift id
1	$id	+id*id$	reduce E → id
2	$E	+id*id$	shift +
3	$E+	id*id$	shift id
4	$E+id	*id$	reduce E → id
5	$E+E	*id$	shift *
6	$E+E*	id$	shift id
7	$E+E*id	$	reduce E→id
8	$E+E*E	$	reduce E→E*E
9	$E+E	$	reduce E→E+E
10	$E	$	accept



LR(k) **L**eft to right 스캔 · **R**ightmost 유도 (역순으로)

감사합니다